

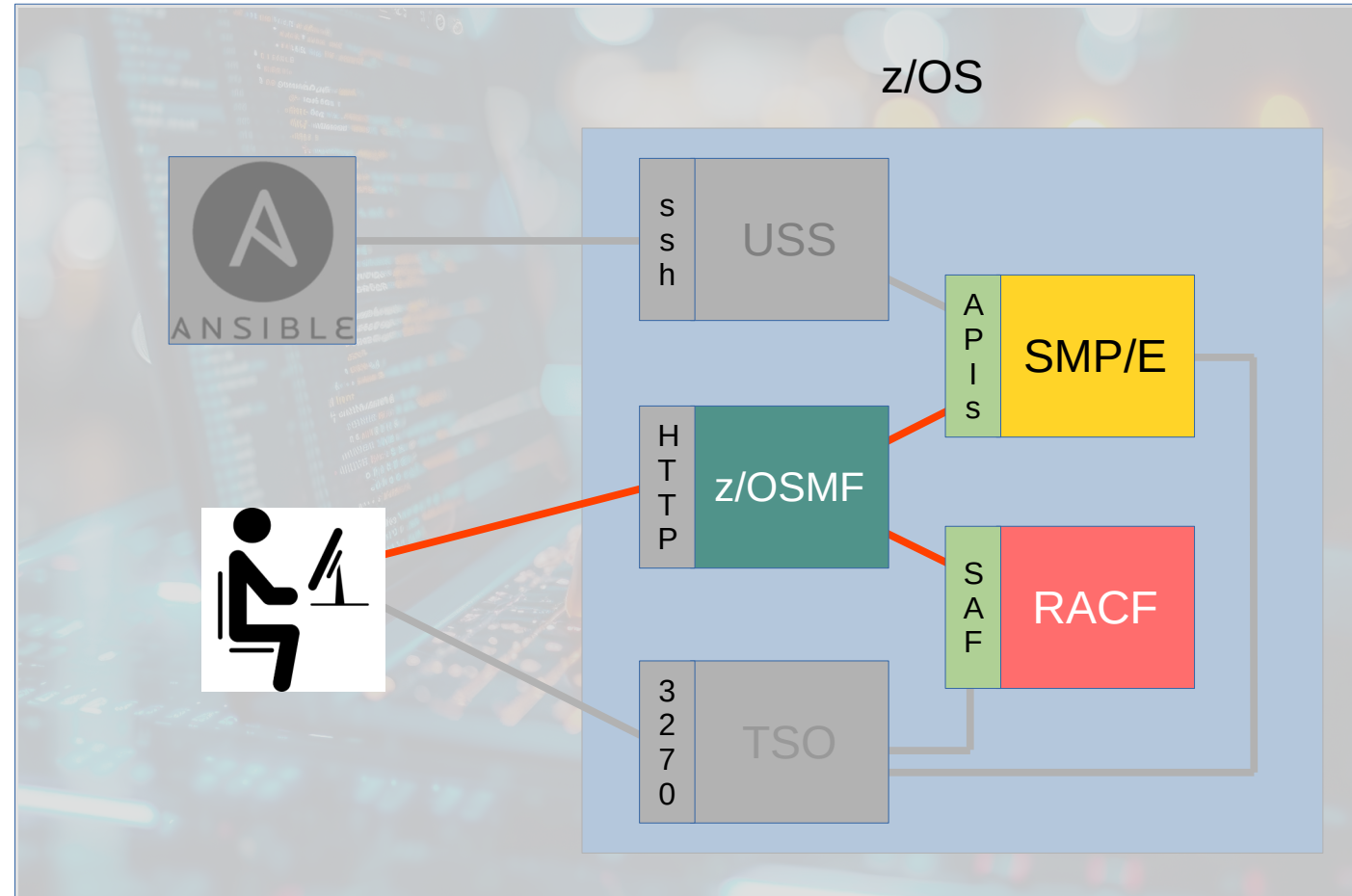
Using Ansible with z/OS Managed Nodes





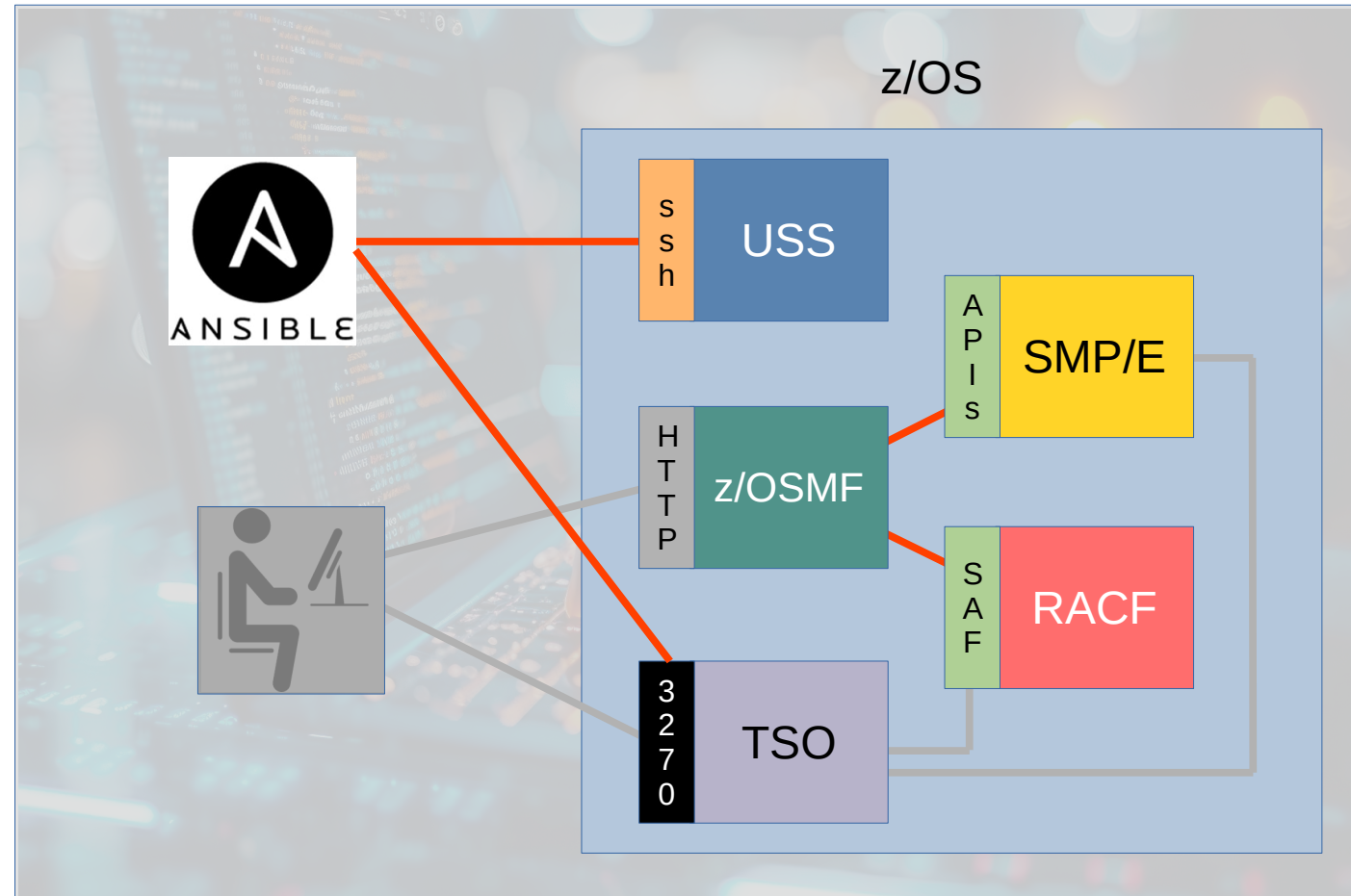
WE WERE WORKING ON THIS

Tools
Userid
Authorization
Data Sets



WE ARE WORKING ON THIS NOW

Tools
Userid
Authorization
Data Sets



SOFTWARE INSTALLATION AND MANAGEMENT

SMP/E

System Modification Program Extended

The package manager for z/OS

z/OSMF

z/OS Management Facility

Browser-based front-end to z/OS system management tools; also exposes APIs used by external tools.

Ansible

An external tool that can interact with z/OSMF via ssh.



We are here

MANAGED NODES ON A Z/OS SYSTEM ARE LPARS

all:

children:

zos:

hosts:

zoslp01:

ansible_host: zoslp01.company.com

sysplex: PLEXA

zoslp02:

ansible_host: zoslp02.company.com

sysplex: PLEXA

zoslp03:

ansible_host: zoslp03.company.com

sysplex: PLEXB

zoslp04:

ansible_host: zoslp04.company.com

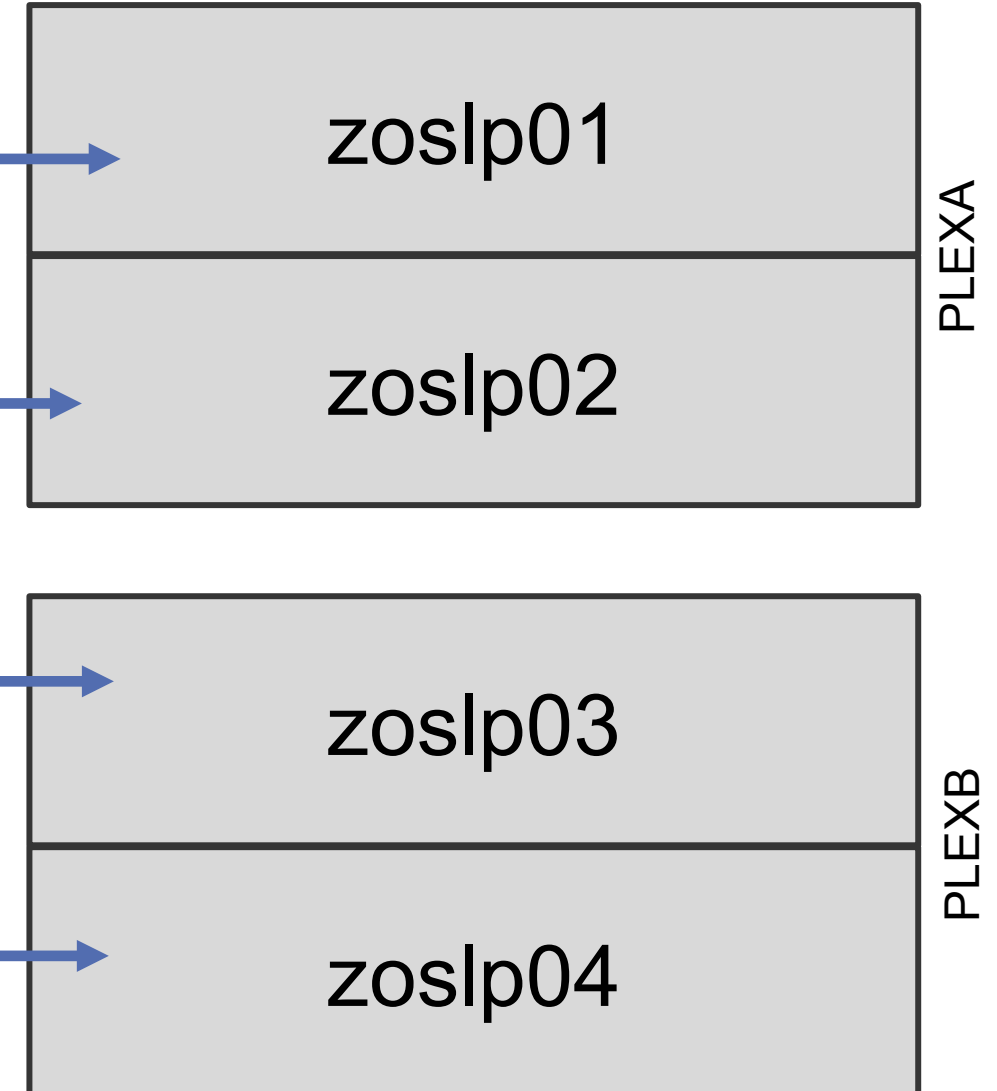
sysplex: PLEXB

vars:

ansible_user: ibmuser

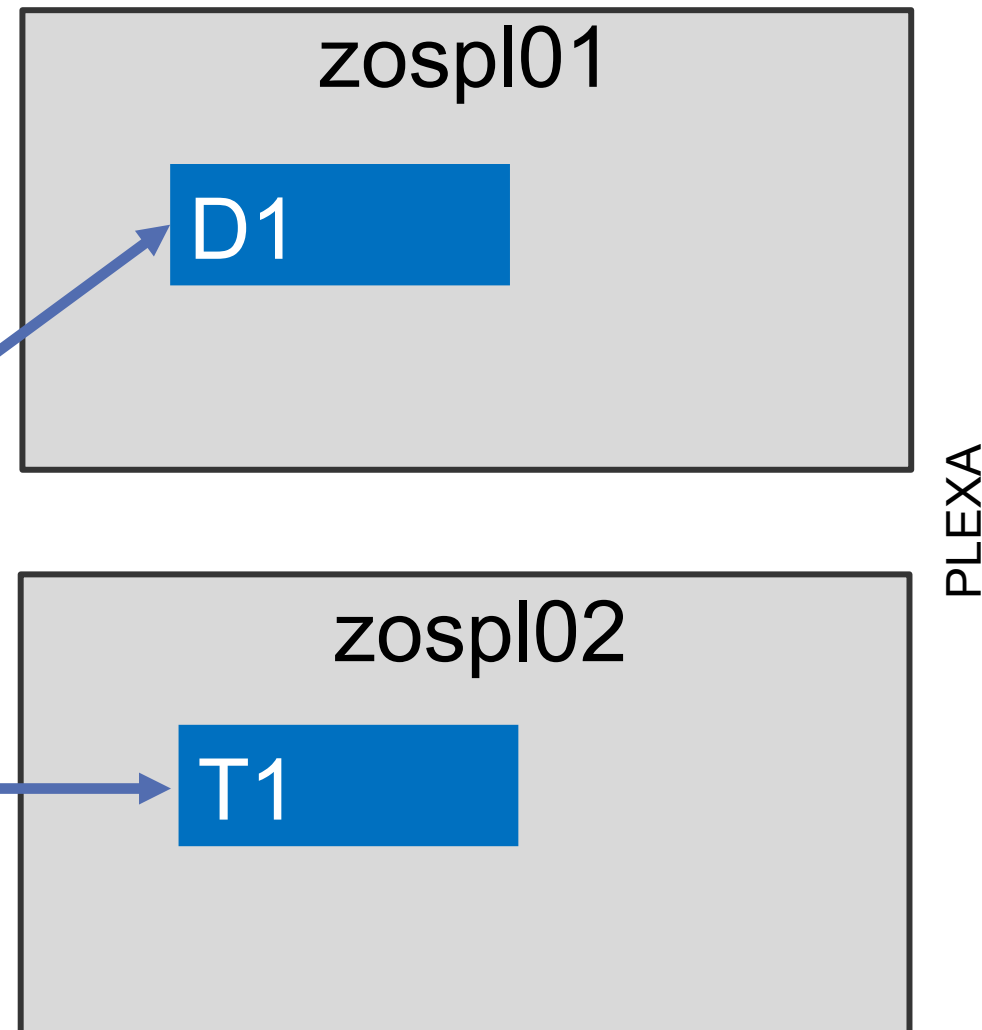
ansible_connection: ssh

ansible_python_interpreter: ...



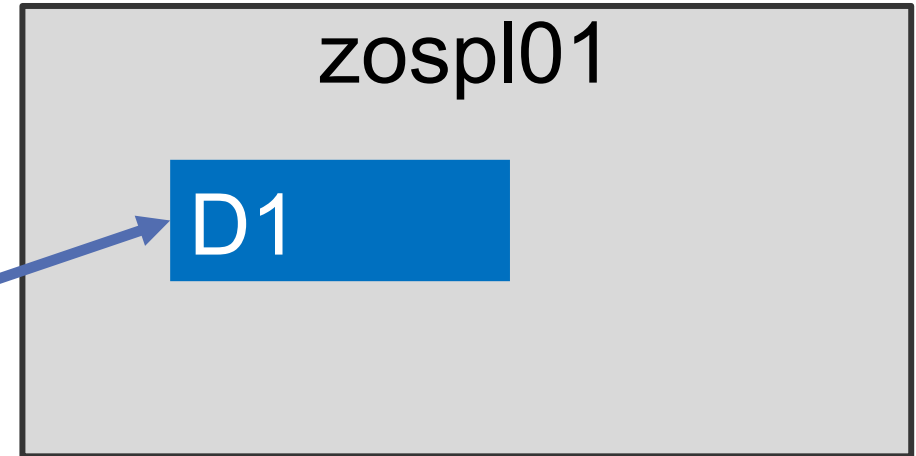
Z/OS SUBSYSTEMS ARE LOGICAL RESOURCES

```
all:
  children:
    zos:
      hosts:
        zoslp01:
          ansible_host: zoslp01.company.com
          sysplex: PLEXA
        zoslp02:
          ansible_host: zoslp02.company.com
          sysplex: PLEXA
      db2_subsystems:
        hosts:
          db2d1:
            zos_host: zospl01
            db2_ssld: D1
            db2_location: D1LOC
          db2t1:
            zos_host: zospl02
            db2_ssld: T1
            db2_location: T1LOC
```



EXAMPLE: STOPPING DB2 SUBSYSTEMS

- name Stop DB2 subsystems
hosts: db2_subsystems
tasks:
 - name: Stop DB2
ibm.ibm_zos_core.zos_operator:
cmd: "STOP DB2,SSID={{ db2_ssid }}"
delegate_to: "{{ zos_host }}"



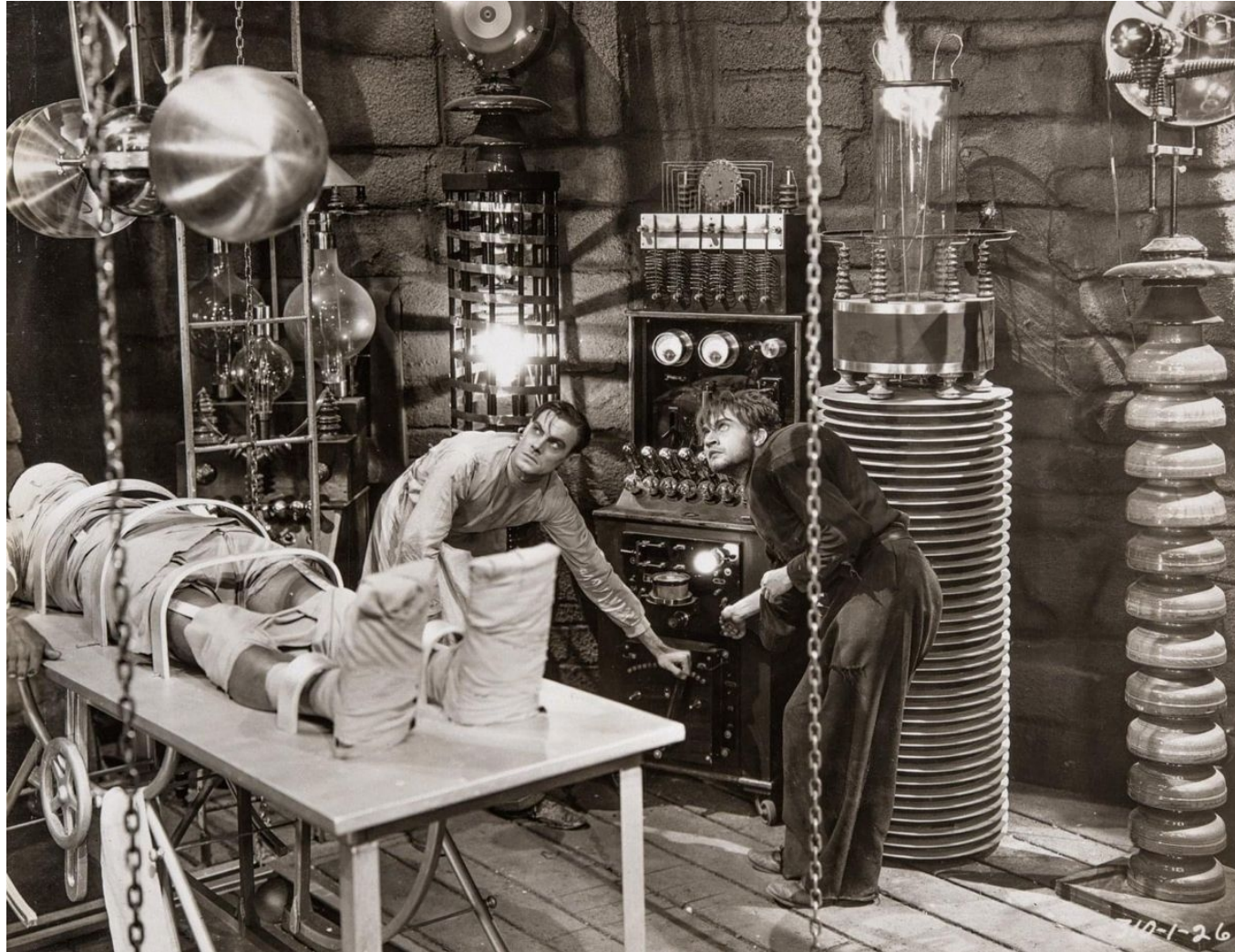
STOP DB2 is a z/OS operator command. `zos_operator` simulates a human operator entering the command on the system console.

The variables come from

```
hostvars[inventory_hostname].zos_host  
hostvars[inventory_hostname].db2_ssid
```

You can see why you have to understand what's happening on the z/OS system. Ansible is only front-ending z/OS functions. If you don't know what those functions are, Ansible will not magically enable you to manage z/OS systems.

LAB Z/OS 39: EXPLORE Z/OS CONFIG FOR ANSIBLE



REQUIREMENTS FOR ANSIBLE AND Z/OS

Ansible and z/OS are both IBM products, but they don't work together seamlessly and transparently "out of the box." There are two reasons for this, in the instructor's opinion.

First, IBM is developing the z/OS Open Automation Utilities (ZOAU) gradually and delivering new functionality with each release. It isn't a mature product yet. That means breaking changes.

Second, IBM acquired Ansible when they acquired Red Hat, who had previously acquired Ansible. The Ansible team does not have a history of following IBM-like or Red Hat-like guidelines for supporting backward compatibility. Still more breaking changes. Really, friends, *a lot* of breaking changes.

Only specific combinations of versions of z/OS, IBM Enterprise Python, ZOAU, Ansible, and the IBM Ansible collections will work. Other combinations will reliably fail in surprising and mysterious ways.

Therefore, the single most useful piece of documentation is the Collection Requirements, which contains dependency matrices for the IBM Ansible collections. Bookmark it. What are you waiting for? Bookmark it.

`https://ibm.github.io/z\_ansible\_collections\_doc/ibm\_zos\_core/docs/source/collection-requirements.html`



DEPENDENCY MATRIX FOR IBM.IBM_ZOS_CORE

At right is a screenshot of the dependency matrix for collection `ibm.ibm_zos_core` as of June 30, 2026. Of course, you aren't expected to read it in detail on a slide. The point is to show that it only goes as far back as version 1.11 of Ansible. The most recent version that will work on our lab z/OS system is 1.9.3. (Discovered through a process of trial and error. Mostly error.)

If a version of Something doesn't work with the version of Something Else that you're using, the error messages will have no relationship with the actual problem. The software is looking for something it needs in the wrong place, or is looking for something that no longer exists. It doesn't know what's going on or how to tell you what it's trying to do.

Once you get the combination of versions right, everything abruptly starts working. *Everything*. You'll feel as if you got cherries all the way across on a slot machine, after punching it ten thousand times.

That is probably the reason most organizations freeze the versions of everything that relates to Ansible as soon as they get things working. It's just too risky to upgrade anything. Many installations remain far behind the current release for a long time.

Version	Control Node	Managed Node
2.1.x	ansible-core >=2.16.x Ansible >=9.0.x AAP >=2.4	z/OS V2R5 - V3Rx z/OS shell z/OS OpenSSH - IBM Open Enterprise SDK for Python - IBM Z Open Automation Utilities >=1.4.1, <1.5.0
2.0.x	ansible-core >=2.16.x Ansible >=9.0.x AAP >=2.4	z/OS V2R5 - V3Rx z/OS shell z/OS OpenSSH - IBM Open Enterprise SDK for Python - IBM Z Open Automation Utilities >=1.4.0, <1.5.0
1.16.x	ansible-core >=2.16.x Ansible >=9.0.x AAP >=2.4	z/OS V2R5 - V3Rx z/OS shell z/OS OpenSSH - IBM Open Enterprise SDK for Python - IBM Z Open Automation Utilities >=1.3.6, <1.4.0
1.15.x	ansible-core >=2.15.x Ansible >=8.0.x AAP >=2.4	z/OS V2R5 - V3Rx z/OS shell z/OS OpenSSH - IBM Open Enterprise SDK for Python - IBM Z Open Automation Utilities >=1.3.5, <1.4.0
1.14.x	ansible-core >=2.15.x Ansible >=8.0.x AAP >=2.4	z/OS V2R5 - V3Rx z/OS shell z/OS OpenSSH - IBM Open Enterprise SDK for Python - IBM Z Open Automation Utilities >=1.3.4, <1.4.0
1.13.x	ansible-core >=2.15.x Ansible >=8.0.x AAP >=2.4	z/OS V2R5 - V3Rx z/OS shell z/OS OpenSSH - IBM Open Enterprise SDK for Python - IBM Z Open Automation Utilities >=1.3.3, <1.4.0
1.12.x	ansible-core >=2.15.x Ansible >=8.0.x AAP >=2.4	z/OS V2R5 - V3Rx z/OS shell z/OS OpenSSH - IBM Open Enterprise SDK for Python - IBM Z Open Automation Utilities >=1.3.2, <1.4.0
1.11.x	ansible-core >=2.15.x Ansible >=8.0.x AAP >=2.4	z/OS V2R4 - V2Rx z/OS shell z/OS OpenSSH - IBM Open Enterprise SDK for Python - IBM Z Open Automation Utilities >=1.3.1, <1.4.0

ANSIBLE INVENTORY

You know an *inventory* is a list of managed nodes that are to be the targets of whatever playbook(s) you launch.

And you know you can group managed nodes that have important things in common.

The canonical examples in Ansible documentation are nodes in a cloud environment, assumed to be virtual Linux instances. They may be grouped geographically, by function, or according to other criteria.

How does this concept translate to the world of mainframe systems?



ANSIBLE INVENTORY

A Parallel Sysplex environment can consist of a 32-machine cluster of IBM Zs, each of which can have up to 85 LPARs running as many as six different operating systems concurrently. Some of the managed nodes may be guest operating systems running under z/VM or KVM; the latter can be cloud environments in their own right.

An Ansible inventory of z/OS managed nodes could, in principle, be pretty extensive. There could be hundreds of nodes with many different configurations.

Or it could be quite simple.

Many organizations don't take full advantage of the potential of the IBM Z. They operate their mainframes as if they were still living in the 1980s, just to keep a handful of mission-critical legacy applications alive. They keep the box out back, by the old fencepost.

In that kind of environment you might only have to manage a couple of LPARs; one for prod and one for dev/test. Or maybe just one for prod.



ANSIBLE INVENTORY

A more robust Sysplex environment may have groups of LPARs that serve different purposes.

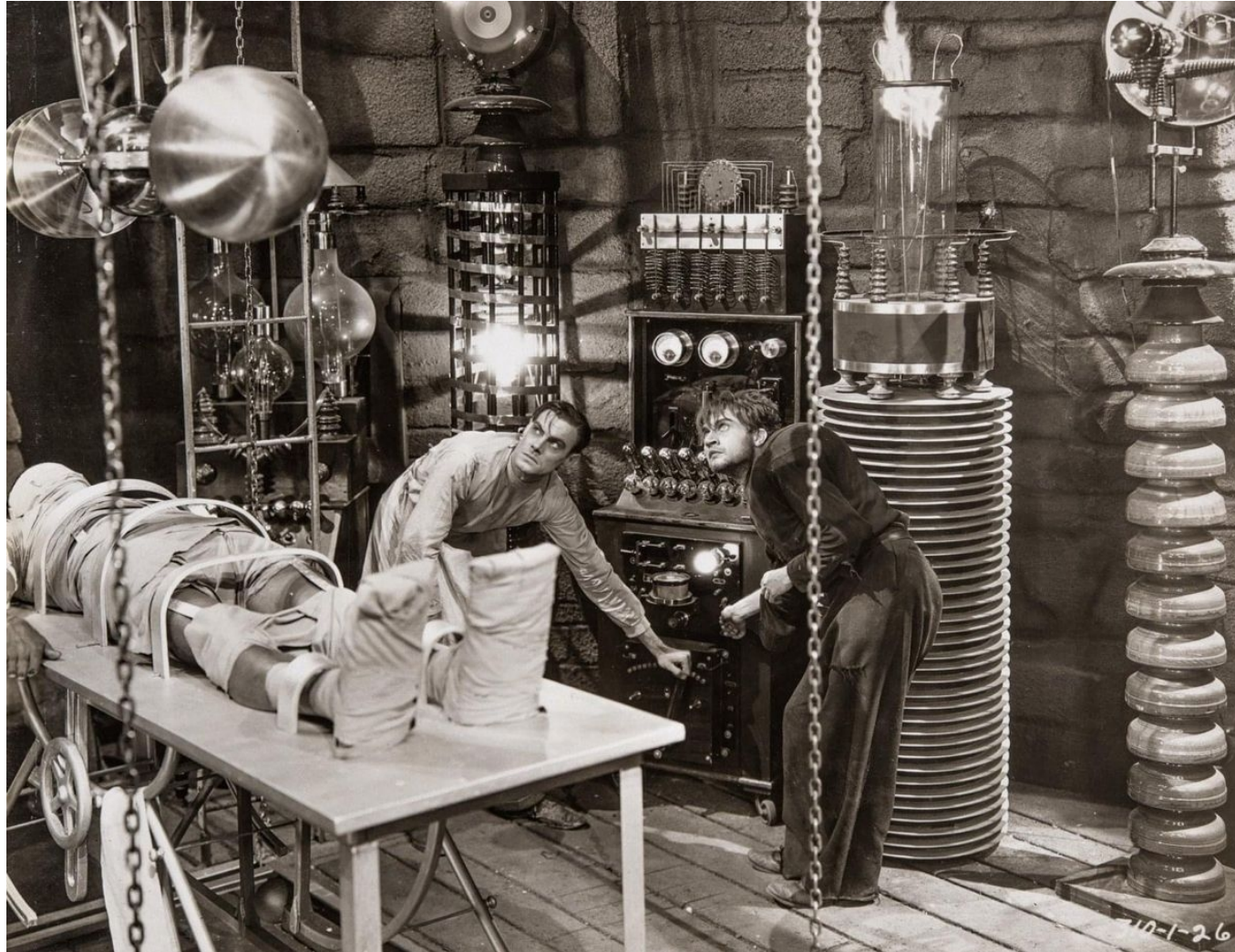
Conceptually, LPARs and other resources may be grouped according to various criteria – production vs. test, high security vs. medium security, geographical location, etc.

Three kinds of groupings are relevant to organizing our Ansible inventories:

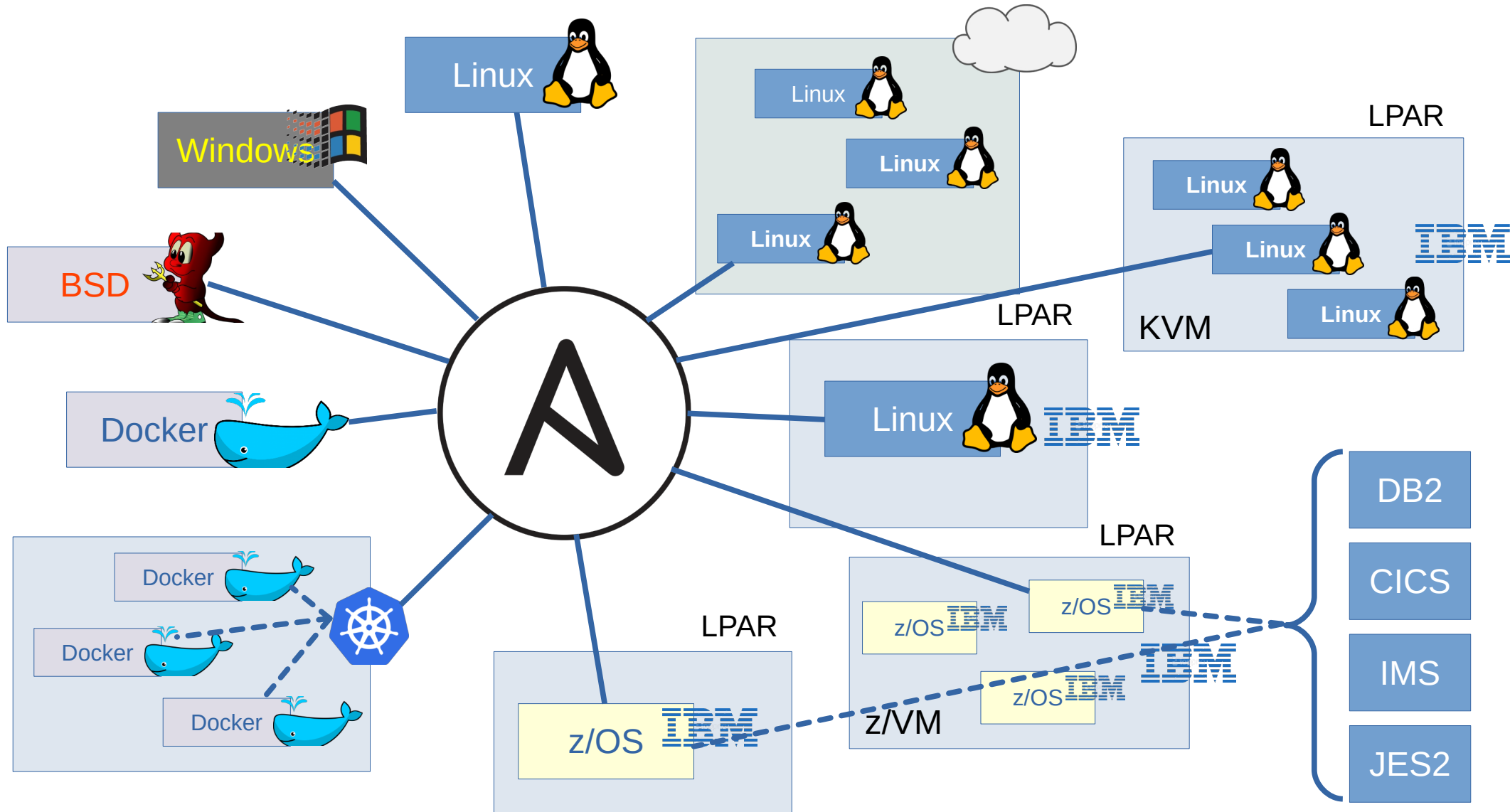
- LPARs with the same configurations; regardless of their function (test, prod, dev, etc.), they are all kept at the same maintenance level and have the same configuration.
- LPARS that can be offline for maintenance at the same time; it's possible not all LPARs that logically belong together can be unavailable to users simultaneously.
- Isolation of functionality to support segregation of duties; it's possible that different teams of platform engineers must take care of prod vs. dev/test LPARs.

```
config1:
  hosts:
    prod1.server.com
    test1.server.com
config2:
  hosts:
    prod2.server.com
    test2.server.com
prod:
  hosts:
    prod1.server.com
    prod2.server.com
test:
  hosts:
    test1.server.com
    test2.server.com
```

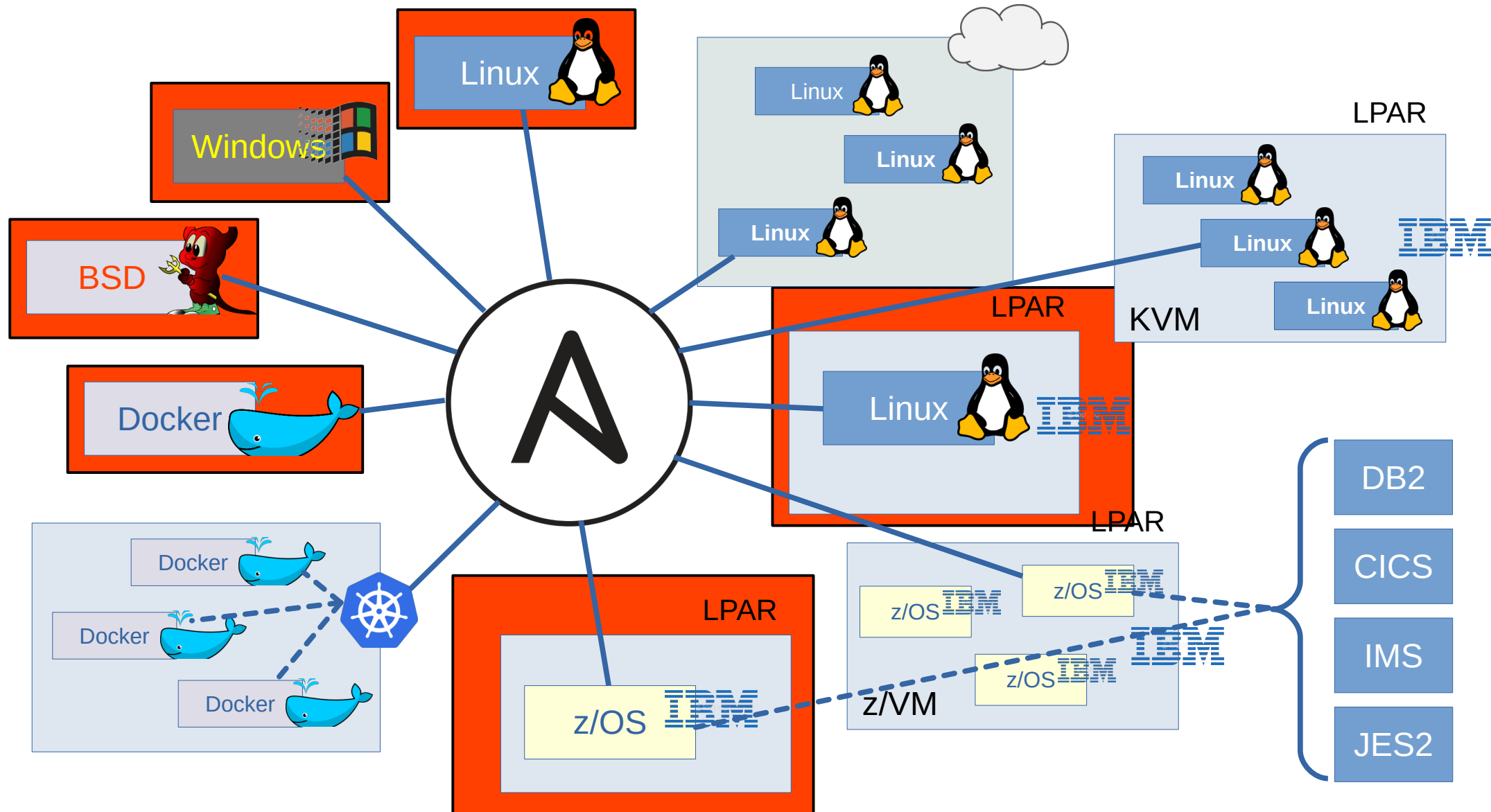

LAB Z/OS 40: CONNECTING ANSIBLE TO Z/OS



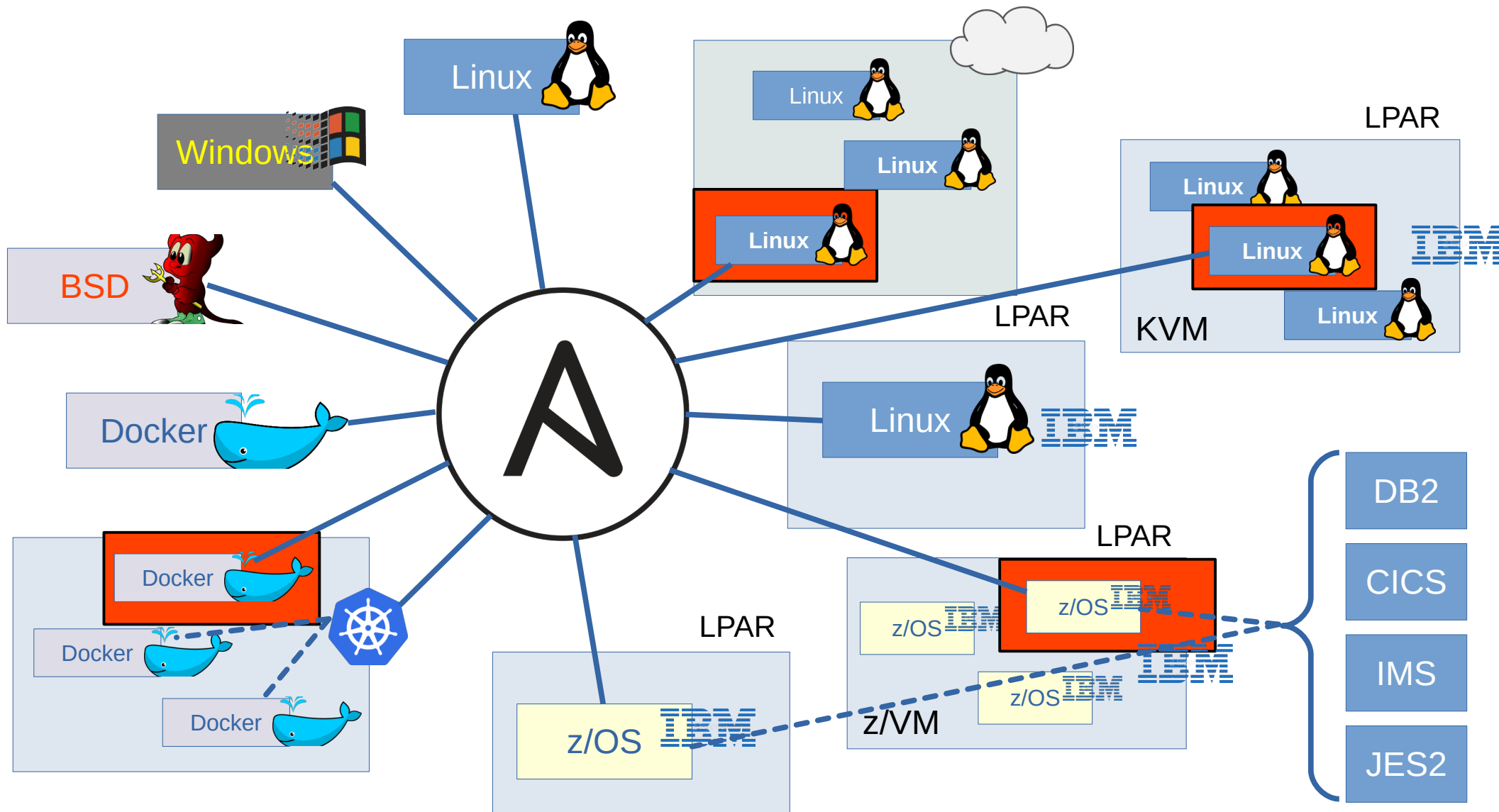
ANSIBLE CONNECTS TO MANAGED ENDPOINTS



ANSIBLE CONNECTS TO SSH ENDPOINTS



ANSIBLE CONNECTS TO SSH ENDPOINTS



ANSIBLE MANAGED NODES

Ansible can work with anything that

- exposes an endpoint
- supports ssh (or WinRM, a local container connection, or a specialized connection plugin)
- has code installed that can process what Ansible sends (on most hosts it's Python; on Windows it's PowerShell)

A managed node can be

- a physical server
- a virtual machine (VM)
- a Linux or Windows container
- a Kubernetes cluster
- a Linux instance on an IBM Z, either running in its own LPAR or as a guest OS under KVM
- a z/OS instance on an IBM Z, either running in its own LPAR or as a guest OS under z/VM
- a network device
- a local execution environment

SPECIAL CASES

Ansible can manage Kubernetes pods, but it does not connect directly to them. It connects to Kubernetes and special code carries out operations on the pods.

```
all:
  children:
    kubernetes:
      hosts:
        prod-cluster:
          kube_api:
            https://api.prod.example.com:6443
          kubeconfig:
            /home/ansible/.kube/prod-config
```

Ansible can manage z/OS subsystems – DB2, IMS, CICS, JES2. It does not connect directly to these facilities. It connects to a z/OS instance defined in its inventory, and special code on z/OS carries out operations on the subsystems.

```
all:
  hosts:
    zos1.company.com:
      ansible_host: 10.1.1.10
      db2_instances:
        - subsystem: DBP1
          location: PROD
        - subsystem: DBP2
          location: QA
      cics_regions:
        - name: CICSA
        - name: CICSB
```

YOU'RE CONNECTING TO Z/OS INSTANCES



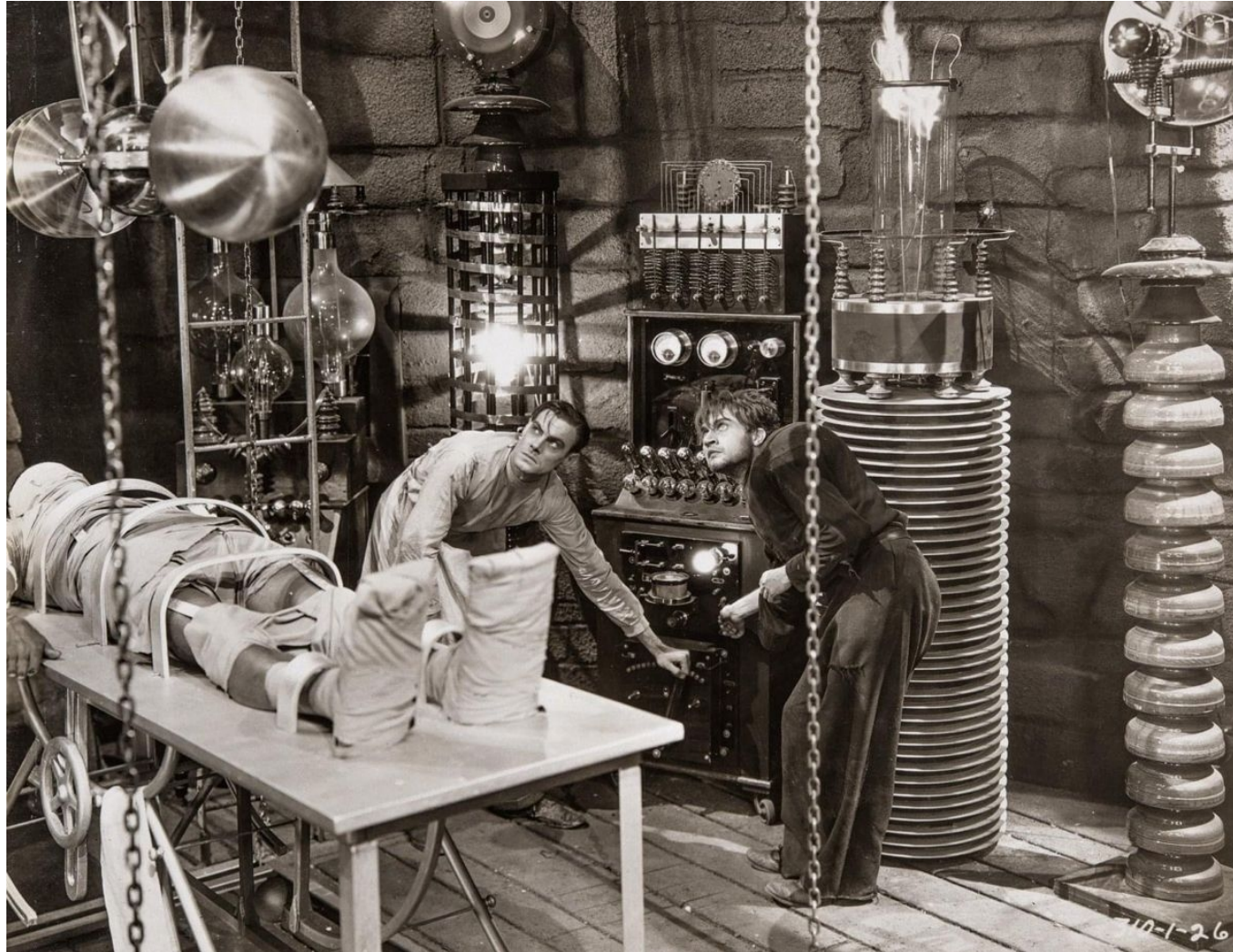
Hey, everyone! I'm connecting to "The Mainframe!"

```
all:
  hosts:
    zos1.company.com:
      ansible_host: 10.1.1.10
      db2_instances:
        - subsystem: DBP1
          location: PROD
        - subsystem: DBP2
          location: QA
      cics_regions:
        - name: CICSA
        - name: CICSB
```

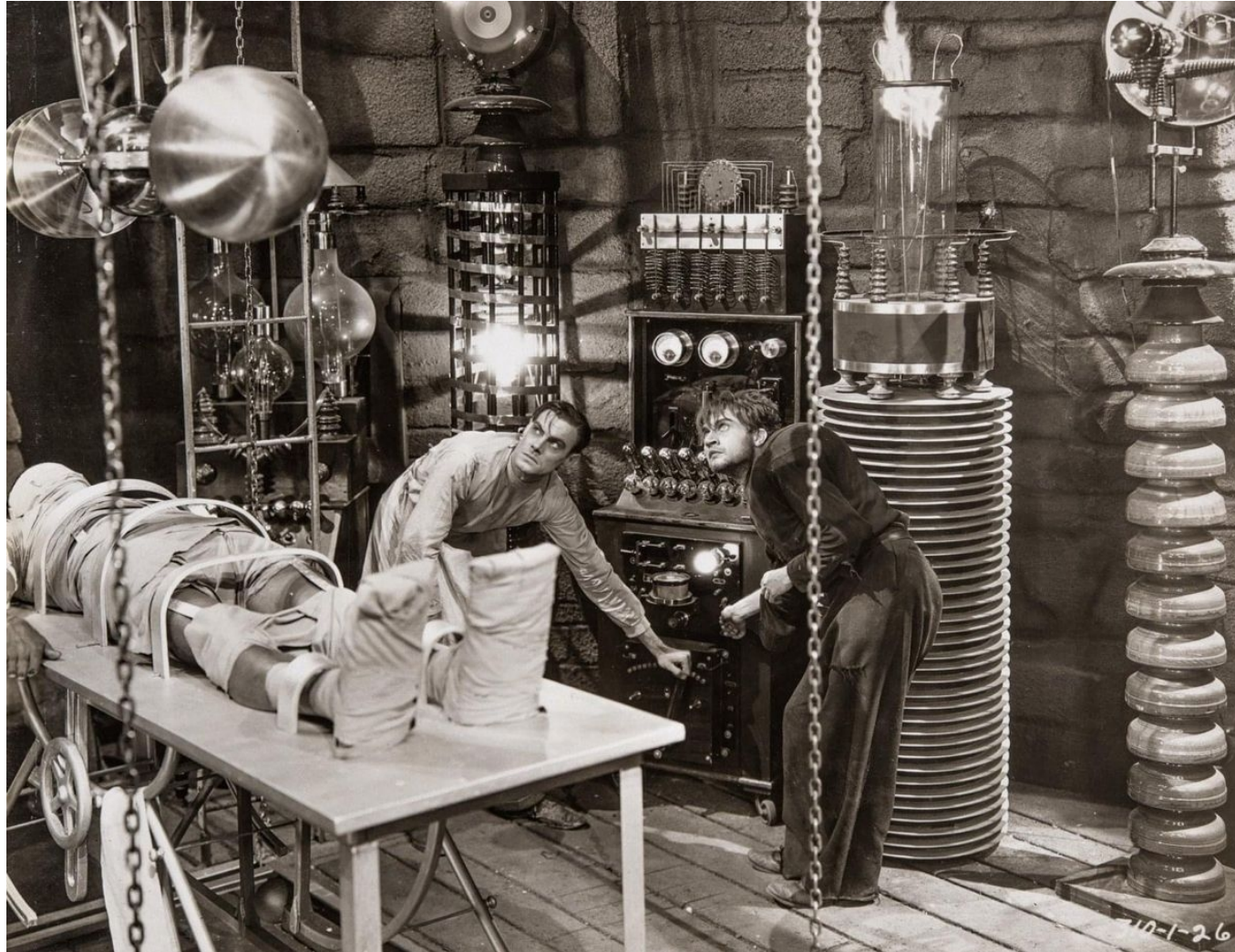


Yesterday he connected to "The AWS!"

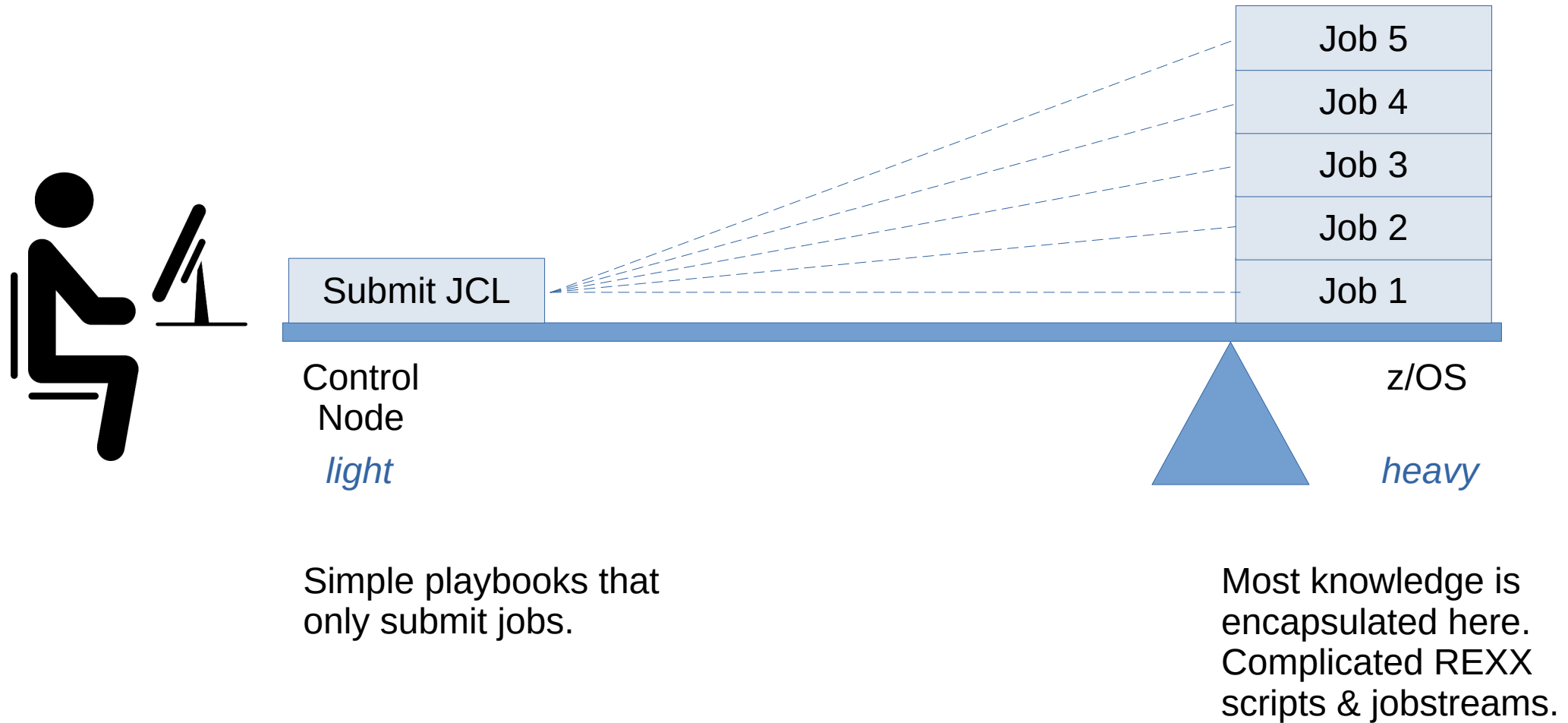
LAB Z/OS 41: ANSIBLE PLAYBOOK TO SUBMIT JCL



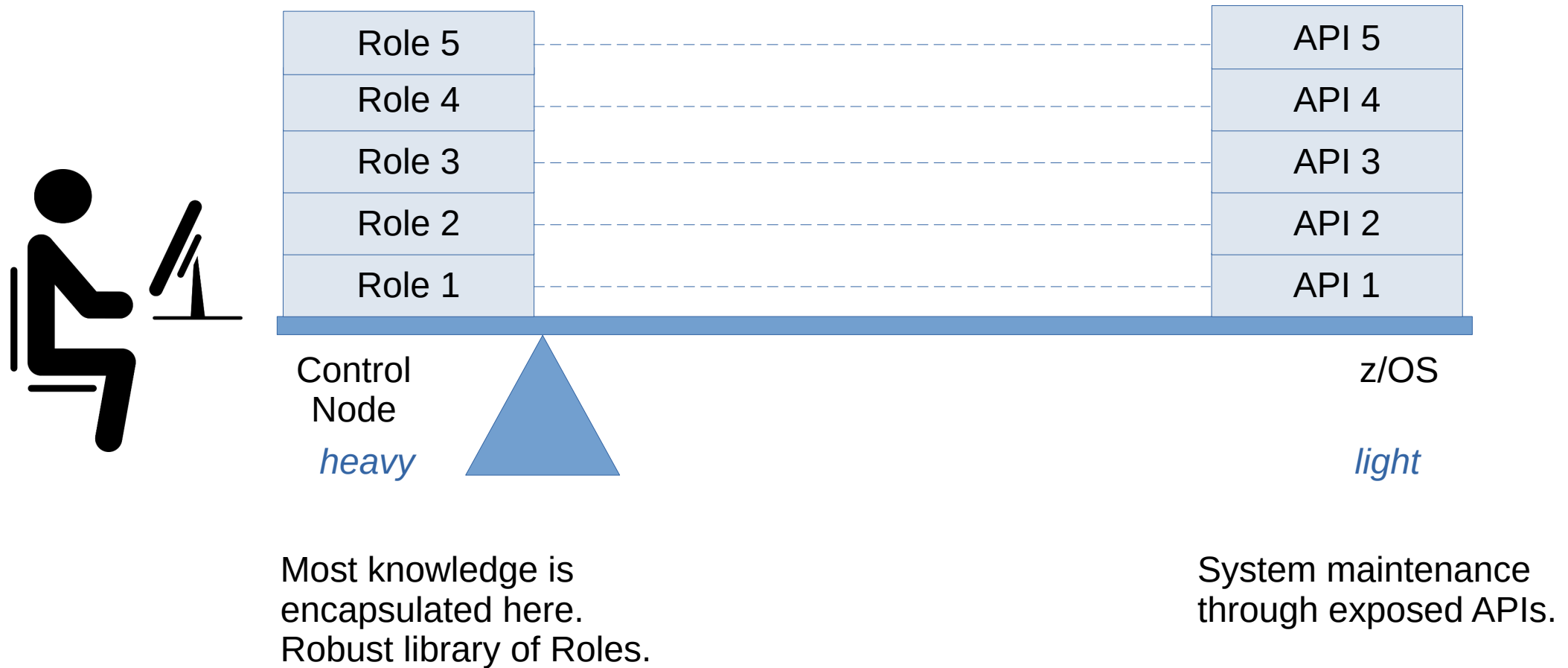
LAB Z/OS 42: SUBMIT PROVISIONING JOB



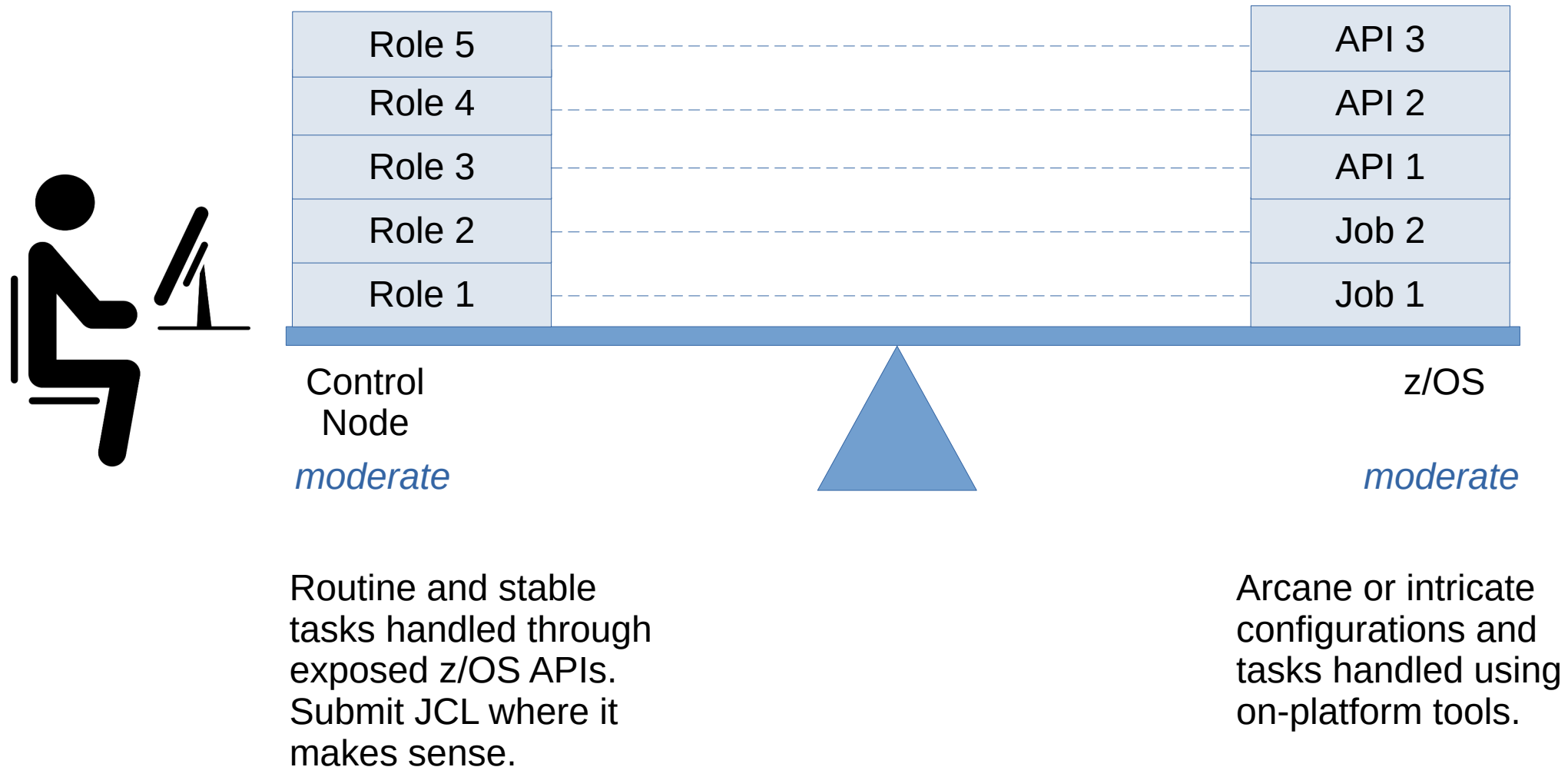
POSITIONING THE FULCRUM



POSITIONING THE FULCRUM



POSITIONING THE FULCRUM



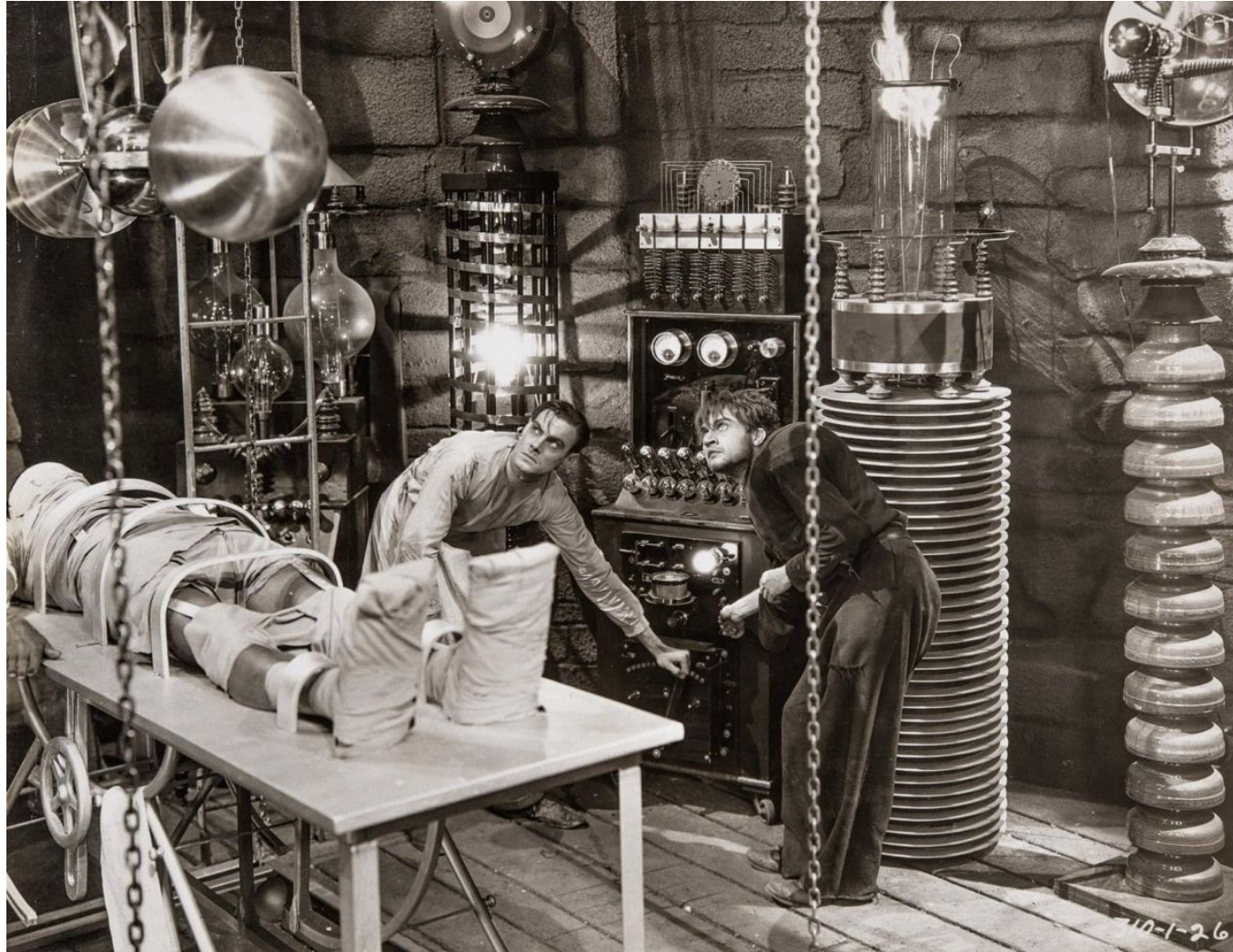
MORE REALISTIC ANSIBLE PROJECT STRUCTURE

The setup for the Ansible project in our initial lab exercises is deliberately simple, so that you won't get bogged down with too many details spread across too many files.

But that is not what real Ansible projects look like. Let's walk through a more realistic directory structure and explain how the information is organized.

```
ubuntu@ip-172-31-44-178:~/ansible-zos$ tree
.
├── README.md
├── ansible.cfg
├── data
│   ├── provdev.jcl
│   │   ├── zoslab
│   │   └── PROVDEV
│   └── tmp
│       └── zoslab
│           └── JOB
├── id_rsa
├── id_rsa.pub
├── inventory
│   ├── group_vars
│   │   └── all
│   │       ├── all.yml
│   │       └── vault.yml
│   ├── host_vars
│   │   └── zoslab.yml
│   └── hosts.yml
├── play
├── playbooks
│   └── roles
│       ├── submit_library_provisioning_job
│       │   ├── tasks
│       │   │   └── main.yml
│       └── zos_dataset_fetch
│           ├── tasks
│           │   └── main.yml
```

LAB Z/OS 43: PROVISION LIBRARIES WITHOUT JCL



WHAT YOU KNOW AND WHAT YOU CAN DO

- You understand how an Ansible project directory structure is likely to look in a "real" environment.
- You can access z/OS with Ansible and run playbooks.
- You can make reasonable judgment calls about where to place more of the complicated logic and decision-making for system maintenance.

WHAT'S NEXT

- Using Ansible with z/OSMF